TKS V8.0: A TYPED FINITE-FUNCTION KERNEL FOR COGNITIVE-ARCHITECTURE SEMANTICS

LITTLE HENDERSON

Abstract. We present a deliberately small, executable reference kernel for the typed-composition layer of the Tootra Knowledge System (TKS) and its secular derivative, the Cognitive Architecture System (CAS). The kernel supplies named finite objects with explicitly ordered carriers; total finite-function arrows with declared source and target and partial, type-checked categorical composition; a hard separation between one-carrier endomaps, which admit monoid powers, and heterogeneous semigroupoid paths, which do not; a deterministic standard-basis lift into finite-dimensional linear operators with an exact operator-norm theorem; and stable denotational normal forms defined by declared generator order and shortlex minimization over a completed finite closure. The kernel is an independent reference model: it does not modify, replace, or certify the active CAS notation compiler, maps no canonical Noetic or CAS construct onto kernel arrows, and makes no empirical cognitive claim. Evidence is precisely bounded: pointwise proofs and bounded exhaustive executable tests (19,683 associativity triples, 289 endomaps through carrier size four, 1,678 lift-functor composition pairs), plus a pinned Lean 4.32.1 formal seed that kernel-checks a named subset of the results under a single standard axiom dependency. Universal formalizations, semantic transfer to CAS, compiler integration, and independent specialist review remain open and are enumerated as explicit proof obligations.

1. Introduction

Prior TKS/CAS formal artifacts (through v7.4 and the v7.5–v7.7 extensions) define a rich ontology—ten Noetics, four Domains, seven Drives, four Registers, and a generative-operation notation governed by the v7.7 operator discipline—but the production notation compiler enforces value *kinds* (state, drive, relation, program) rather than categorical *source and target objects*. Three failure modes follow, each observed during the Project Coherence mathematical red-team audit:

(1) **Untyped composition.** Nothing in a kind system prevents composing two operations whose intermediate objects disagree, silently producing an expression with no denotation.
(2) **Invalid power inference.** A sequence of heterogeneous, time-varying generators is not an element of one finite semigroup; treating an arbitrary generator sequence as one operator iterated n times licenses false fixed-point and idempotence conclusions.
(3) **Nondeterministic identity.** When carrier order is left to hash order, basis indices, denotations, and canonical representatives silently depend on interpreter state, which destroys replayability.

Date: August 7, 2026.

White paper v0.1, 2026-08-07. Artifact status: DRAFT / GATE 1 PROTOTYPE / RING 4 / NOT LICENSED / NOT FROZEN. Drafting this paper promotes no claim. Source basis: `TKS_V8_CORE_FORMAL_SPEC_AND_PROOF_OBLIGATIONS_v0.1.md` (SHA-256 `F740BA05...`), prototype manifest v0.1.0, implementation `src/tks_v8_core/`.

TKS v8.0 answers these with the smallest kernel that makes each failure a type error, while explicitly refusing to over-claim: the kernel is a semantic substrate that *future* TKS/CAS constructs may be mapped onto after review, not a formalization of the existing canon.

2. The typed finite-function category

Definition 2.1 (Objects)**.** An object is a pair $X = (\mathrm{name}_X, \mathrm{carrier}_X)$ where $\mathrm{carrier}_X$ is a finite, explicitly ordered sequence of distinct hashable elements. Object identity includes the declared name as well as the carrier.

Including the name in identity prevents accidental coercion between semantically distinct objects that happen to have equal cardinality or equal elements. All order-sensitive inputs (carrier elements, arrow mappings, path arrows, word labels, catalog generators, lift coordinate vectors) reject sets, mappings, and one-shot iterables: callers must provide a replayable ordered sequence, because order changes basis indices, denotations, and shortlex rank.

Definition 2.2 (Arrows)**.** An arrow judgment $f : X \to Y$ is a total map from every standard source index of X to one standard target index of Y. Two arrows are extensionally equal when source, target, and the full mapping tuple agree; presentation names do not affect equality.

Definition 2.3 (Identity and composition)**.** For each object X, $\mathrm{id}_X(i) = i$. Given $f : X \to Y$ and $g : Y \to Z$,

$$(g \circ f)(i) = g(f(i)) : X \to Z.$$

The implementation rejects $g \circ f$ unless the declared target of f *equals* the declared source of g; no cardinality-only or carrier-only coercion is performed.

Proposition 2.4 (Category laws)**.** *Left identity, right identity, and associativity hold for typed total finite functions under Definition 2.3.*

Proof. Pointwise: $(f \circ \mathrm{id}_X)(i) = f(i)$, $(\mathrm{id}_Y \circ f)(i) = f(i)$, and $((h \circ g) \circ f)(i) = h(g(f(i))) = (h \circ (g \circ f))(i)$ for all standard indices i. □

Remark 2.5. The executable suite exercises these laws exhaustively on a bounded family (19,683 three-point associativity triples), and a pinned Lean module kernel-checks the identity and associativity lemmas exactly. Bounded exhaustive testing is not proof-assistant verification of the Python implementation; the Python-to-Lean equivalence remains open.

2.1. Endomaps versus semigroupoid paths.

The kernel's central discipline separates two constructions that informal notation tends to conflate.

Definition 2.6 (Endomap)**.** Endomap(f) is constructible only when $f : X \to X$. It admits the monoid power f^n for $n \geq 0$, with $f^0 = \mathrm{id}_X$.

Definition 2.7 (Semigroupoid path)**.** A semigroupoid path is a nonempty sequence

$$X_0 \xrightarrow{f_1} X_1 \xrightarrow{f_2} \cdots \xrightarrow{f_n} X_n$$

whose adjacent object types match. It may cross carriers and has a typed composite, but it *intentionally has no power operation.*

The missing power operation is the point: it blocks the invalid inference from arbitrary typed or time-varying generator sequences to one finite semigroup element.

Theorem 2.8 (Safe finite power)**.** *For a finite endomap $f : X \to X$, the power $f^{|X|!}$ is idempotent.*

Proof. For $|X| \geq 2$: after $|X|!$ iterations every point of the functional graph of f has entered its cycle, and every possible cycle length $\ell \leq |X|$ divides $|X|!$; a further $|X|!$ iterations therefore leaves every value unchanged, so $f^{|X|!} \circ f^{|X|!} = f^{|X|!}$. The empty and one-point cases are immediate. □

Remark 2.9. This is a standard finite-semigroup result; the kernel contributes an implementation tested exhaustively for carriers through size four (289 endomaps). Its Lean formalization remains open (Obligation PO-04). Exponentiation by squaring uses bounded internal presentation names (bit length plus a short SHA-256 label for large exponents), so name construction cannot exceed the exponent's bit length; regression tests cover 2^{30} and 10^{5000}.

3. The deterministic standard-basis lift

Definition 3.1 (Lift)**.** For $f : X \to Y$, let H_X and H_Y be the finite-dimensional Hilbert spaces with orthonormal bases indexed by X and Y. The lift is the linear map

$$A_f : H_X \to H_Y, \qquad A_f\, e_x = e_{f(x)}.$$

Theorem 3.2 (Exact operator norm)**.** *For nonempty X, with the empty-domain value defined as zero,*

$$\|A_f\|^2 \;=\; \max_{y \in Y} \left|f^{-1}(y)\right|.$$

Proof. The (x, x') entry of $A_f^* A_f$ equals 1 exactly when $f(x) = f(x')$. Grouping the input basis by fibers, this Gram matrix is a direct sum of all-ones blocks J_m, one per nonempty fiber. The eigenvalues of J_m are m (once) and 0 (multiplicity $m - 1$). Hence the largest eigenvalue of $A_f^* A_f$ is the maximum fiber size, which is $\|A_f\|^2$. □

Proposition 3.3 (Implemented consequences)**.** $\operatorname{tr}(A_f^* A_f) = |X|$*;* $\operatorname{rank}(A_f) = |\operatorname{im}(f)|$*; and A_f is an isometry if and only if f is injective.*

The kernel keeps three operators distinct: $A_f^* A_f$ is the input-collision Gram matrix; $A_f A_f^*$ is diagonal with fiber sizes; and the codomain support projector is the separate diagonal 0/1 matrix marking nonempty fibers.

Remark 3.4 (Permanent counterexample fixture). For the constant map on two points,

$$A_f = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix},$$

the lift is idempotent yet has norm $\sqrt{2}$, trace-Gram 2, rank 1, and is neither a partial isometry nor an orthogonal projector. This fixture permanently blocks the tempting but false general claims "deterministic lifts are contractive" and "idempotent lifts are projectors."

Proposition 3.5 (Functoriality)**.** *The lift satisfies $A_{\mathrm{id}_X} = I_{H_X}$ and $A_{g \circ f} = A_g A_f$.*

Remark 3.6. Functoriality is checked by exact matrix equality for every valid typed-map pair on carriers through size three (1,678 composition pairs, including empty carriers). The Lean seed additionally kernel-checks identity entrywise and on basis vectors, typed composition on basis vectors, and relational-product support composition. Finite-sum numerical matrix multiplication and arbitrary-vector linearity remain open.

4. Stable denotational normal forms

Definition 4.1 (Shortlex normal form)**.** Let $G = (g_0, \dots, g_{k-1})$ be a finite, explicitly ordered set of typed generators. A typed word carries a start object, and its labels must compose in sequence. Shortlex compares word length first, then the declared generator rank sequence. For a reachable finite-function denotation a,

$$\mathrm{NF}_G(a) \;=\; \text{the shortlex-least well-typed word } w \text{ with } [\![w]\!] = a.$$

The empty word at each object denotes its identity.

Proposition 4.2 (Soundness of pruning; finiteness)**.** *The closure algorithm that explores words in shortlex order and expands only the first representative of each extensional denotation computes* NF_G *on the completed closure, which is finite.*

Proof sketch. If two prefixes have the same typed denotation, appending the same well-typed suffix preserves both denotational equality and the shortlex advantage of the earlier prefix; pruning the later prefix therefore never removes a least representative. Finiteness holds because for finite objects X, Y there are at most $|Y|^{|X|}$ total maps $X \to Y$ and the object set is finite. □

Remark 4.3 (Boundaries). The implementation returns only after closure completion, deduplicates queued candidates by typed denotation, and raises a resource error rather than ever returning a partial table as canonical. Two boundaries are explicit: (i) this is denotational minimization over a completed finite closure, *not* a claim that any inherited TKS rewrite system terminates or is confluent; (ii) changing the declared generator order may change the selected representative without changing its denotation.

5. Formal verification status

A pinned, isolated Lean 4.32.1 toolchain kernel-checks a named subset of the results. The verifier hardcodes the sole allowed standard axiom (`propext`), rejects project axiom declarations and `sorry`, checks source hashes, builds through Lake, and audits exact theorem dependency reports.

Verified scope: typed total-function identities and associativity; endomap-only powers; the exact two-state constant-lift expansion and noncontractivity counterexample (Remark 3.4); deterministic-lift identity entrywise and on basis vectors; typed basis composition; relational-product support composition.

Open scope: the universal lift-norm theorem (Theorem 3.2); the generic bridge from the two-state fixture to matrix, Gram, trace, and rank definitions; finite-sum numerical matrix multiplication; arbitrary-vector linear extension; Theorem 2.8; shortlex leastness; any presheaf, sheaf, or product-site formalization.

6. Proof and integration ledger

ID	Obligation	Status
PO-01	Identity/associativity of composition	partial formal verification present
PO-02	Reject every source/target mismatch	executable validation
PO-03	Powers only for endomaps	executable validation
PO-04	Theorem 2.8 formalization	standard result; formalization open
PO-05	Universal norm formula (Thm. 3.2)	informal proof present; formalization open
PO-06	Shortlex soundness and leastness	informal argument; formalization open
PO-07	Independent theorem-prover formalization	partial, present
PO-08	Reconcile kernel types with corrected v7.4 records	open; not licensed
PO-09	Integrate with the active notation compiler	open
PO-10	Independent specialist review	open
PO-11	Lift functor laws (Prop. 3.5)	partial formal verification; matrix bridge open

7. Relation to the wider program

The kernel is the finite-arrow boundary object for three downstream candidates described in companion papers. *Noetic Topos 2.0* builds its marked context-factor site over a legacy category whose future reconciliation with kernel objects is Obligation PO-08. *CAS PCBA 8* may adapt only a certified total, finite, deterministic, stateless component transition to a kernel arrow; chains become semigroupoid paths, and feedback powers require a certified endomap. *CAS Core 8* uses the kernel for legacy fractal-digit composition checks with an explicit caller-supplied catalog of typed arrows, failing closed on missing digits or source/target mismatches. In each case the kernel supplies type-correct composition only; no adapter carries an automatic CAS semantic theorem, and mapping the ten canonical Noetics or any CAS construct onto kernel arrows remains ring-4 and not licensed pending review.

8. What TKS v8.0 does not claim

Excluded and still open: parsing or evaluating TKS v7.4/v8 surface syntax; mapping canonical Noetics or CAS constructs to kernel arrows; RPM monad data or laws; a rewrite system for inherited TKS equations; Noetic Topos 2.0 coverage and modalities; stochastic kernels, quantum channels, or empirical dynamics; and any transfer from the Ten Advances manuscript. No empirical, cognitive-dynamics, channel-contractivity, or CAS-efficacy claim follows from any result above.

9. Reproduction

From the repository root with `src` on `PYTHONPATH`:

```
python -m unittest discover -s tests/tks_v8_core -v
python scripts/verify_tks_v8_gate1.py
```

The 2026-08-04 validation snapshot records PASS on 20 tests with the bounded exhaustive case counts cited in the abstract. Passing establishes only the behavior covered by this finite

executable model; it does not close the open formalization, integration, semantic-transfer, or review obligations.

10. Conclusion

TKS v8.0 contributes a disciplined foundation rather than a sweeping theory: typed composition that cannot silently mistype, powers that exist only where a monoid exists, a lift whose exact norm is proved rather than assumed, and canonical forms that are deterministic because their inputs are ordered. The kernel's refusal to encode the TKS/CAS ontology before review is deliberate: the mapping of canon onto substrate is itself a gated research obligation, and preserving that boundary is part of the result.

References

[1] Project Coherence, *Mathematical Red-Team Report*, v0.1.
[2] Project Coherence, *Theorem Ledger*, v0.1: records THM-LIFT-NORM-FIBER-FORMULA, THM-LIFT-OCCUPATION-COUNTEREXAMPLE, THM-FINITE-IDEMPOTENT-POWER.
[3] *Lean Proof Manifest*, v0.1: exact scope of the pinned Lean 4.32.1 seed.
[4] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998. Background for the categorical laws exercised; no novelty claimed.